Theme: Personal Expense Tracker DB

Section A: Concept Application

1. Explain how relational databases help maintain accuracy in expense records.

- Relational databases keep expense record accurate by organizing data into connected tables and applying rules.
  They use keys to link data correctly, so every expense matches a valid user or category.
  Constraints stops the wrong entries.
  Normalizations avoid duplicate or inconsistent data.
  Also transaction ensures all changes are saved properly or not at all.
  This structures helps prevent errors and keeps data inconsistent.

2. Why are constraints important in personal finance data?

- Constraints are important in personal financial data because they prevent incorrect or invalid entries.
  They make sure things like:
  The amount are not negative
  Data are valid
  Required fields (like category) are not empty

  This keeps the data are accurate, consistent and reliable , which is essential for tracking expenses correctly and making good financial decisions.

3. How does GROUP BY help analyze spending patterns?

- GROUP BY helps you summarize expenses by grouping similar data.
  How it helps analyze spending:
  - **Category-wise Spending:-**
    you can group expenses by category
    Query:- SELECT category, SUM(amount) FROM expenses GROUP BY category;

  - **Monthly spending spends:-**

Query:- SELECT Month(date), SUM(amount) FROM expenses GROUP BY Month(date);

- **Frequent expense types:-**
  You can count how often you spend in each category.
  Query:- SELECT category, COUNT(*) FROM expenses GROUP BY category;

- **Average Spending behavior:-**
  Query:- SELECT category, AVG(amount) FROM expenses GROUP BY category;

4. Explain a scenario where rollback is required during expense entry.

➢ A rollback is needed when something goes wrong while entering an expense and you don't want partial or incorrect data saved.

Example scenario:-
Suppose you are adding new expenses :-
Amount: 500
Category: Food
Payment Method : card

The system start saving the data,
But the category is invalid or
The payment update fails.

What Rollback Does:-
Rollback cancels the entire transactions and restore the database to it's previous state, so no partial or wrong data is saved.

5. How do views help users track monthly expenses efficiently?

➢ View helps users track monthly expenses by saving ready-made query that shows only the needed data.

A view can show expenses grouped by month.
It hides complex SQL, so user just query the view.
It always shows updated data automatically.

Query:- SELECT Month(date) AS month, SUM(amount) total FRey OM expenses GROUP BY Month(date);

6. Why use triggers for automatic category or balance updates?

- Triggers are used to automatically updated data when an expenses added, updated and deleted.

Why they are useful:-

Auto Category Handling :- Assign or adjust categories without manual input.

Auto Balance Update :- Deduct amount from balance as soon as an expense is entered.

Data Consistency :- ensures related data is always correct.

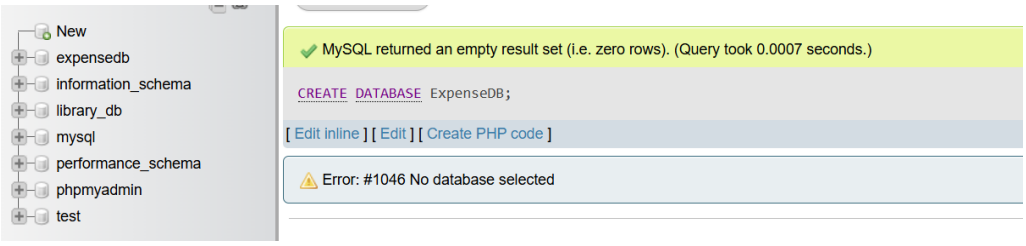
Less Manual Work:- no need to write extra queries every time.

Section B: SQL Hands-On

1. **Create a Database on ExpenseDB;**

⇒ Create database ExpenseDB;

Screenshot:-



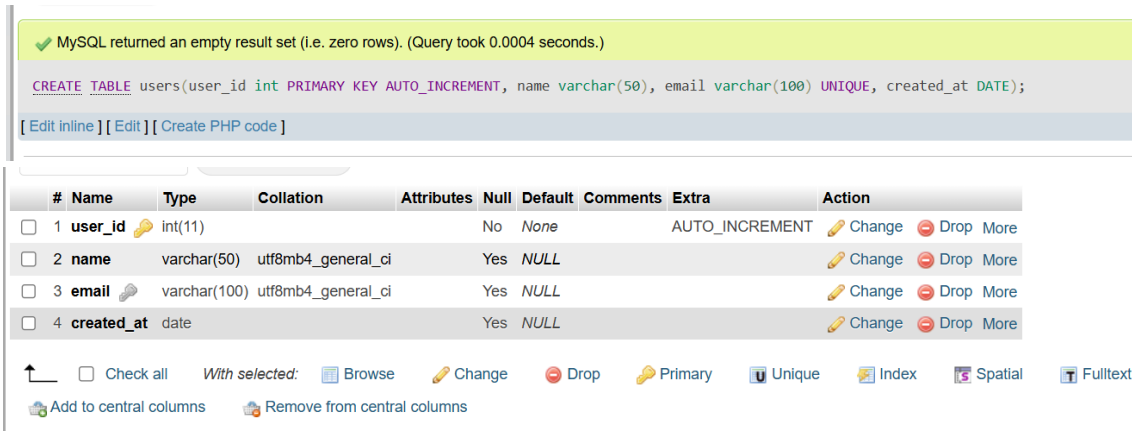
2. **Create a Table for users, categories and expenses.**

⇒ Below the table are created.

➢ Table 1:- Users

CREATE TABLE users(user_id int PRIMARY KEY AUTO_INCREMENT, name varchar(50),email varchar(100) UNIQUE, created_at DATE);

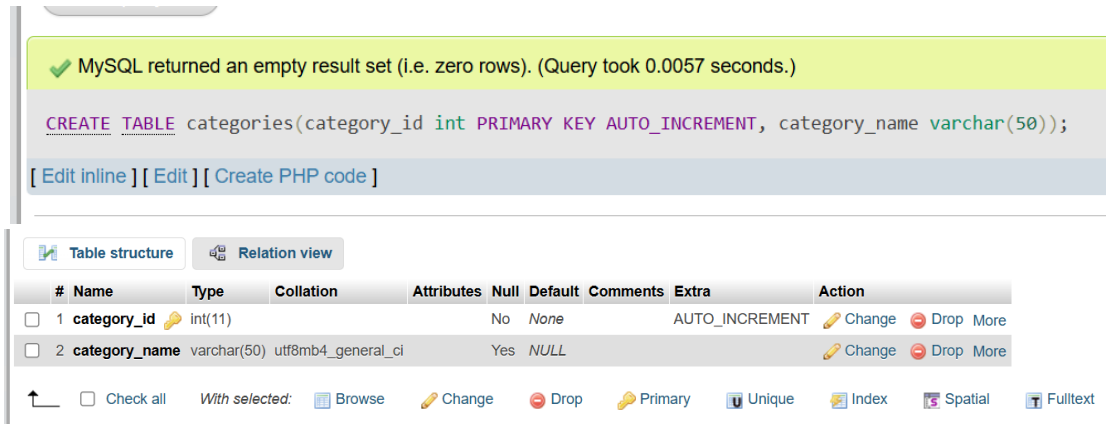
Screenshot:-



➢ Table 2:- Categories

CREATE TABLE categories(category_id int PRIMARY KEY AUTO_INCREMENT, category_name varchar(50));

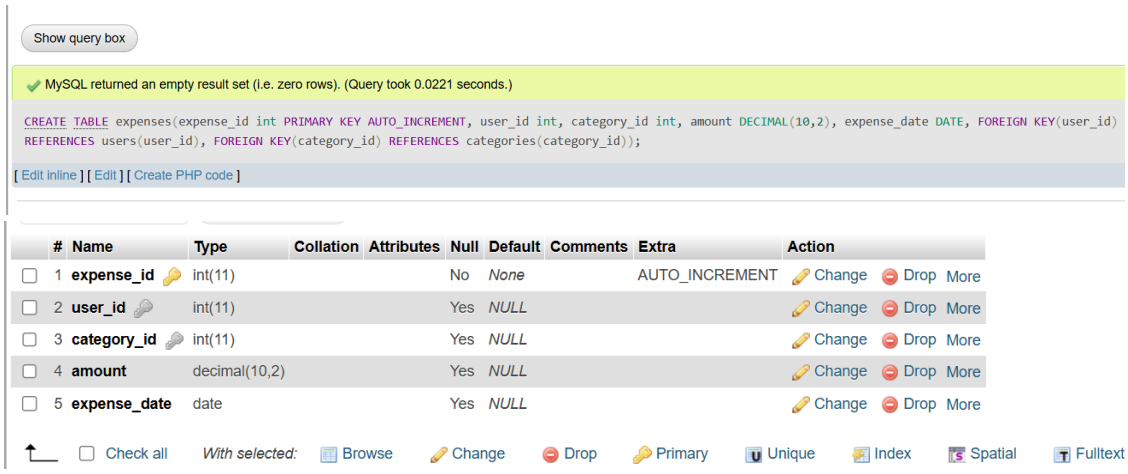
Screenshot:-



- Table 3:- Expenses

CREATE TABLE expenses(expense_id int PRIMARY KEY AUTO_INCREMENT, user_id int, category_id int, amount DECIMAL(10,2), expense_date DATE, FOREIGN KEY(user_id) REFERENCES users(user_id), FOREIGN KEY(category_id) REFERENCES categories(category_id));

Screenshot:-



➢ DDL Understanding :-

1. Explain why foreign keys are used: Specifically, why must user_id in the expenses table correspond to a real ID in the users table?

⇒ Foreign keys are used to enforce relationships and maintain data integrity between tables and database.
In your personal expense tracker example,
The users table store valid users.
The expense table records transactions and includes a user_id columns.

Why must user_id in expenses correspond to a real ID in users?
Because of referential integrity – a core concept in relational database.

✓ Prevent Invalid data
✓ Ensures meaningful relationships
✓ Maintain consistency
✓ Improves query reliability.

2. Mention one issue that would occur if foreign keys were removed: Describe the risk of "Orphaned Records" (e.g., an expense existing for a user who has been deleted).

⇒ In foreign keys are removed, one major issue is orphaned records.
This happens when expense remains in the expense table even though its's related user has been deleted from the users table

Risk:

- The expense has a user_id that no longer exists.
- It becomes unclear who expense belongs to
- Reports and query may show incorrect or confusing data.

➢ DML Operations:-

Write SQL queries to:

● **Insert 5 users into the users table with unique names and emails.**

INSERT INTO users(name,email,created_at)VALUES('YashTailor','yash165@gmail.com','2026-04-24');

INSERT INTO users(name,email,created_at)VALUES('Raj Patel','raj15@gmail.com','2026-02-18');

INSERT INTO users(name,email,created_at)VALUES('Sahil Patel','sahil56@gmail.com','2025-01-12');

INSERT INTO users(name,email,created_at)VALUES('KrunalRathod','krunal12@gmail.com','2024-04-22');

INSERT INTO users(name,email,created_at)VALUES('ParthPatel','parth23@gmail.com','2023-02-07');

Screenshot:-

←T→				user_id	name	email	created_at
☐	Edit	Copy	Delete	1	Yash Tailor	yash165@gmail.com	2026-04-24
☐	Edit	Copy	Delete	2	Raj Patel	raj15@gmail.com	2026-02-18
☐	Edit	Copy	Delete	3	Sahil Patel	sahil56@gmail.com	2025-01-12
☐	Edit	Copy	Delete	4	Krunal Rathod	krunal12@gmail.com	2024-04-22
☐	Edit	Copy	Delete	5	Parth Patel	parth23@gmail.com	2023-02-07

Check all *With selected:* Edit Copy Delete Export

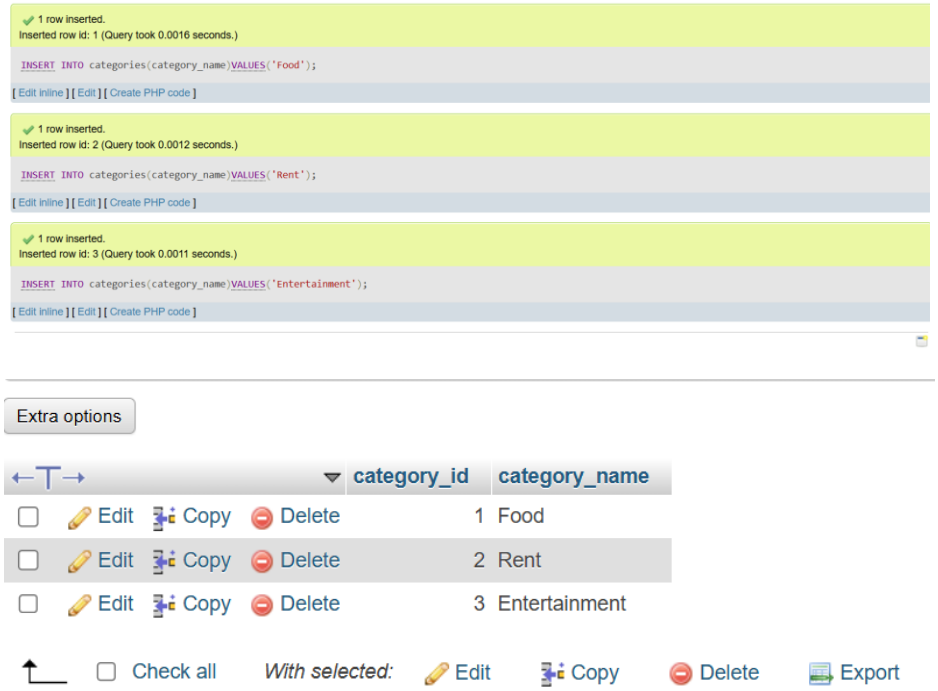
- **Insert 3 categories (e.g., 'Food', 'Rent', 'Entertainment').**

INSERT INTO categories(category_name)VALUES('Food');

INSERT INTO categories(category_name)VALUES('Rent');

INSERT INTO categories(category_name)VALUES('Entertainment');

Screenshot:-



- **Insert 10 expense records distributed across different users and categories.**
  INSERT INTO expenses (expense_id, user_id, category_id, amount, expense_date)
  VALUES
  (1, 1, 1, 250.50, '2024-06-01'),
  (2, 1, 2, 8000.00, '2024-06-02'),
  (3, 2, 1, 300.00, '2024-06-03'),
  (4, 2, 3, 1200.00, '2024-06-04'),
  (5, 3, 2, 7500.00, '2024-06-05'),
  (6, 3, 1, 150.75, '2024-06-06'),
  (7, 4, 3, 500.00, '2024-06-07'),
  (8, 4, 1, 220.00, '2024-06-08'),
  (9, 5, 2, 9000.00, '2024-06-09'),
  (10, 5, 3, 650.00, '2024-06-10');

←T→	expense_id	user_id	category_id	amount	expense_date
Edit Copy Delete	1	1	1	250.50	2024-06-01
Edit Copy Delete	2	1	2	8000.00	2024-06-02
Edit Copy Delete	3	2	1	300.00	2024-06-03
Edit Copy Delete	4	2	3	1200.00	2024-06-04
Edit Copy Delete	5	3	2	7500.00	2024-06-05
Edit Copy Delete	6	3	1	150.75	2024-06-06
Edit Copy Delete	7	4	3	500.00	2024-06-07
Edit Copy Delete	8	4	1	220.00	2024-06-08
Edit Copy Delete	9	5	2	9000.00	2024-06-09
Edit Copy Delete	10	5	3	650.00	2024-06-10

Check all *With selected:* Edit Copy Delete Export

- **Update one incorrect expense: Use the UPDATE command to change the amount of a specific expense_id.**

  UPDATE expenses SET amount = 200.75 WHERE expense_id=6;

1 row affected. (Query took 0.0022 seconds.)

```
UPDATE expenses SET amount = 200.75 WHERE expense_id=6;
```

[Edit inline] [Edit] [Create PHP code]

Edit Copy Delete	6	3	1	200.75	2024-06-06

- **Delete one expense: Remove a record where the amount is less than a specific value.**

  DELETE FROM expenses WHERE amount < 500;

4 rows deleted. (Query took 0.0037 seconds.)

```
DELETE FROM expenses WHERE amount < 500;
```

[Edit inline] [Edit] [Create PHP code]

	expense_id	user_id	category_id	amount	expense_date
Edit Copy Delete	2	1	2	8000.00	2024-06-02
Edit Copy Delete	4	2	3	1200.00	2024-06-04
Edit Copy Delete	5	3	2	7500.00	2024-06-05
Edit Copy Delete	7	4	3	500.00	2024-06-07
Edit Copy Delete	9	5	2	9000.00	2024-06-09
Edit Copy Delete	10	5	3	650.00	2024-06-10

Check all *With selected:* Edit Copy Delete Export

➢ Data Retrieval:-

- **Display all expenses with details: Show the expense_date, amount, name (from users), and category_name (from categories) using INNER JOIN.**

  Query:-

  SELECT expenses.expense_date, expenses.amount, users.name, categories.category_name FROM expenses INNER JOIN users ON expenses.user_id = users.user_id INNER JOIN categories ON expenses.category_id = categories.category_id;

expense_date	amount	name	category_name
2024-06-02	8000.00	Yash Tailor	Rent
2024-06-04	1200.00	Raj Patel	Entertainment
2024-06-05	7500.00	Sahil Patel	Rent
2024-06-07	500.00	Krunal Rathod	Entertainment
2024-06-09	9000.00	Parth Patel	Rent
2024-06-10	650.00	Parth Patel	Entertainment

- **Show total expense amount per category: Use SUM(amount) and GROUP BY category_name.**

  Query:-

  SELECT categories.category_name, SUM(expenses.amount) AS total_expense FROM expenses INNER JOIN categories ON expenses.category_id = categories.category_id GROUP BY categories.category_name;

Extra options

category_name	total_expense
Entertainment	2350.00
Rent	24500.00

- **Display users sorted by total spending: List user names and their total sum of expenses, ordered from highest to lowest.**

  Query:-

  SELECT users.name, SUM(expenses.amount) AS total_spending FROM expenses INNER JOIN users ON users.user_id = expenses.user_id GROUP BY users.name ORDER BY total_spending DESC;

name	total_spending 1
Parth Patel	9650.00
Yash Tailor	8000.00
Sahil Patel	7500.00
Raj Patel	1200.00
Krunal Rathod	500.00

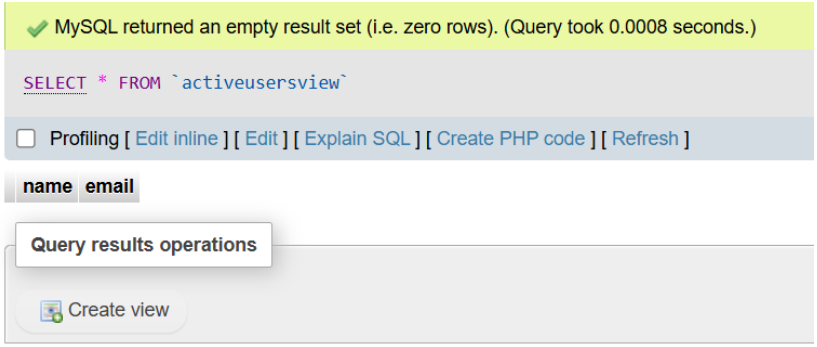
➢ Views :-

- **Create a view named ActiveUsersView: This view should list the name and email of any user who has logged more than 5 individual expense records.**

  Query:-

  CREATE VIEW ActiveUsersView AS SELECT users.name, users.email FROM users INNER JOIN expenses ON users.user_id = expenses.user_id GROUP BY users.user_id, users.name, users.email HAVING COUNT(expenses.expense_id) > 5;

SELECT *FROM ActiveUsersView;



Section C: Mini Project

Mini Project: Expense Tracker DB

Using the same schema:

- **Write CRUD queries**

1. **Insert :- (Create)**

Query:-

INSERT INTO expenses(expense_id, user_id, category_id, amount, expense_date)values(11, 1, 1, 450.00, ''2024-06-11'');

1 row inserted. (Query took 0.0016 seconds.)

```
INSERT INTO expenses(expense_id,user_id,category_id,amount,expense_date)VALUES(11,1,1,450.00,'2024-06-11');
```

[Edit inline] [Edit] [Create PHP code]

←T→			expense_id	user_id	category_id	amount	expense_date
Edit	Copy	Delete	2	1	2	8000.00	2024-06-02
Edit	Copy	Delete	4	2	3	1200.00	2024-06-04
Edit	Copy	Delete	5	3	2	7500.00	2024-06-05
Edit	Copy	Delete	7	4	3	500.00	2024-06-07
Edit	Copy	Delete	9	5	2	9000.00	2024-06-09
Edit	Copy	Delete	10	5	3	650.00	2024-06-10
Edit	Copy	Delete	11	1	1	450.00	2024-06-11

2. **Select(Read):-**

Query:-

SELECT *FROM expenses;

←T→			expense_id	user_id	category_id	amount	expense_date
Edit	Copy	Delete	2	1	2	8000.00	2024-06-02
Edit	Copy	Delete	4	2	3	1200.00	2024-06-04
Edit	Copy	Delete	5	3	2	7500.00	2024-06-05
Edit	Copy	Delete	7	4	3	500.00	2024-06-07
Edit	Copy	Delete	9	5	2	9000.00	2024-06-09
Edit	Copy	Delete	10	5	3	650.00	2024-06-10
Edit	Copy	Delete	11	1	1	450.00	2024-06-11

3. Update :-

Query:-

UPDATE expenses SET amount = 800.65 WHERE expense_id = 10;

1 row affected. (Query took 0.0024 seconds.)

```
UPDATE expenses SET amount = 800.65 WHERE expense_id = 10;
```

[Edit inline] [Edit] [Create PHP code]

	Edit	Copy	Delete	10	5	3	800.65	2024-06-10

4. Delete :-

Query:-

DELETE FROM expenses WHERE expense_id = 11;

1 row deleted. (Query took 0.0023 seconds.)

```
DELETE FROM expenses WHERE expense_id = 11;
```

[Edit inline] [Edit] [Create PHP code]

←T→				expense_id	user_id	category_id	amount	expense_date
	Edit	Copy	Delete	2	1	2	8000.00	2024-06-02
	Edit	Copy	Delete	4	2	3	1200.00	2024-06-04
	Edit	Copy	Delete	5	3	2	7500.00	2024-06-05
	Edit	Copy	Delete	7	4	3	500.00	2024-06-07
	Edit	Copy	Delete	9	5	2	9000.00	2024-06-09
	Edit	Copy	Delete	10	5	3	800.65	2024-06-10

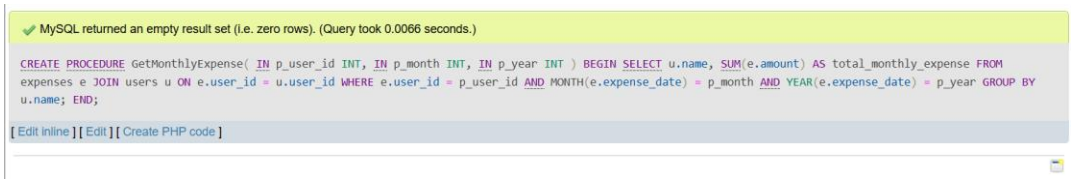
- **Write a stored procedure to calculate monthly user expense**

Query:-

```
DELIMITER //
CREATE PROCEDURE GetMonthlyExpense(
   IN p_user_id int,
   IN p_month int,
   IN p_year int
)
BEGIN
    SELECT users.name, SUM(expenses.amount) AS total_monthly_expense FROM
expenses JOIN users ON expenses.user_id = users.user_id WHERE  expenses.user_id =
p_user_id AND MONTH(expenses.expense_date) = p_month AND
YEAR(expenses.expense_date) = p_year GROUP BY users.name
END //

DELIMITER ;
```

Execute Query:- CALL GetMonthlyExpense(1, 6, 2024);



name	total_monthly_expense
Yash Tailor	8000.00

- **Demonstrate COMMIT and ROLLBACK with example queries**

ROLLBACK :-

Query:-

```
START TRANSACTION;
```

INSERT INTO expenses(expense_id,user_id,category_id,amount,expense_date)VALUES(12,2,1,600.00,'2024-06-12');

SELECT *FROM expenses WHERE expense_id = 12;

ROLLBACK;

	expense_id	user_id	category_id	amount	expense_date
Edit Copy Delete	2	1	2	8000.00	2024-06-02
Edit Copy Delete	4	2	3	1200.00	2024-06-04
Edit Copy Delete	5	3	2	7500.00	2024-06-05
Edit Copy Delete	7	4	3	500.00	2024-06-07
Edit Copy Delete	9	5	2	9000.00	2024-06-09
Edit Copy Delete	10	5	3	800.65	2024-06-10

Check all *With selected:* Edit Copy Delete Export

COMMIT :-

Query :-

START TRANSACTION;

INSERT INTO expenses(user_id,category_id,amount,expense_date)VALUES(3,2,700.00,'2024-06-13');

COMMIT;

	expense_id	user_id	category_id	amount	expense_date
Edit Copy Delete	2	1	2	8000.00	2024-06-02
Edit Copy Delete	4	2	3	1200.00	2024-06-04
Edit Copy Delete	5	3	2	7500.00	2024-06-05
Edit Copy Delete	7	4	3	500.00	2024-06-07
Edit Copy Delete	9	5	2	9000.00	2024-06-09
Edit Copy Delete	10	5	3	800.65	2024-06-10
Edit Copy Delete	13	3	2	700.00	2024-06-13

Click the drop-down arrow to toggle column's visibility.